



National Autonomous University of Mexico

Faculty of Engineering

Computer Engineering

Compilers

JACARANDA LEXICAL ANALYZER REPORT

Team 01 – Group 5

Semester 2027-1

Team members: Aguilar Lara Ericka - 321227122
Bautista Reyes Sofía - 321103891
Cruz Basilio Ximena Carolina - 321116424
Garibay Zamorano Josué Benjamín - 321206105
Jardón Marín Rodrigo - 321240693

Mexico City, Mexico. October 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Motivation	3
1.3	Objectives	3
2	Theoretical Framework	4
2.1	Lexical Analyzer	4
2.2	Token and Lexeme	4
2.3	Regular Expressions	5
2.4	Finite Automata	6
2.5	Lexical Errors	6
2.6	Context-Free Grammar and Left Factoring	7
3	Development	7
3.1	General Project Logic	7
3.2	Project Organization and Data Flow	8
3.3	Graphical User Interface	9
3.4	Finite Automata Design	10
3.4.1	Keyword Automaton	10
3.4.2	Identifier Automaton	12
3.4.3	Operator Automaton	12
3.4.4	Punctuation Automaton	13
3.4.5	Numeric Constants Automaton	14
3.4.6	Boolean Constants Automaton	14
3.4.7	String Literal Automaton	14
3.5	Relationship Between Automata and Regular Expressions	15
3.6	Lexer Implementation	15
3.7	Whitespace and Case Sensitivity	16
3.8	Input and Error Handling	16
3.9	CFG Proposal and Left Factoring	16

3.10	Test Cases	17
4	Results	18
4.1	Required Declaration Example	19
4.2	String and Keyword Example	19
4.3	Switch and Colon Test	21
4.4	Forty-Token Test	21
4.5	Lexical Error Test	23
4.6	File Input Test	23
5	Repository	24
6	Conclusions	24

1 Introduction

1.1 Problem Statement

Lexical analysis is the first stage in which source text is transformed into meaningful units before its syntactic structure is examined. Instead of sending isolated characters directly to the parser, the lexer groups them into tokens such as keywords, identifiers, operators, constants, and punctuation symbols. This transformation produces a cleaner representation of the program that can later be used by the remaining compiler stages.

For this project, the team was asked to develop a lexical analyzer capable of reading either a string or a text file, identifying the tokens present in the input, displaying their classification, and reporting the total number of tokens. The analyzer must recognize the examples established in the activity, including the keyword **print**. In addition, a basic context-free grammar must be proposed for one of the analyzed examples and left factoring must be applied to that grammar.

The project is presented under the name **Jacaranda**. Jacaranda represents the team company responsible for developing and presenting the analyzer as a small software product that can later be extended with the syntactic and semantic stages studied during the course.

1.2 Motivation

A compiler cannot efficiently process a source program if every later stage has to repeatedly interpret raw characters. Lexical analysis solves this problem by assigning a category to each relevant sequence in the input. For example, **int** can be recognized as a keyword, **counter** as an identifier, **=** as an operator, and **20** as a constant. The resulting sequence of tokens becomes the input for syntactic analysis and provides a structured representation of the source program [1, 2].

This activity also connects several concepts studied in class. Regular expressions provide a formal description of lexical patterns, finite automata show how those patterns can be recognized through states and transitions, and the implementation converts the theoretical rules into a working program. Because this is the first analyzer of the project, it also establishes a modular base that can be extended during the semester.

1.3 Objectives

General objective. Develop a lexical analyzer that receives source text from a string or file, recognizes the lexemes contained in the input, classifies them into the required token categories, and reports the complete token sequence and its total count.

Specific objectives.

- Recognize keywords, identifiers, operators, constants, and punctuation symbols.
- Recognize `print` as a keyword and correctly process the examples required in the activity.
- Define lexical rules with regular expressions and represent their recognition logic using finite automata.
- Accept source input directly from the terminal or from a text file.
- Ignore whitespace and supported single-line comments because they do not generate tokens.
- Detect lexical errors when a sequence cannot be recognized by any defined rule.
- Propose a basic context-free grammar and apply left factoring to remove a common prefix.
- Keep the source code, examples, and documentation organized in the public repository of Team 01.

2 Theoretical Framework

2.1 Lexical Analyzer

A lexical analyzer, also known as a *lexer* or *scanner*, reads an input sequence and groups characters into lexemes. Each lexeme is associated with a token type that describes its role in the language. The resulting token sequence can then be consumed by a parser or another compiler component [1].

In Jacaranda, the lexical rules are designed by the team and implemented in Python using RPLY. RPLY provides the scanning mechanism that associates regular-expression patterns with token names [3]. The library does not define the language by itself: the supported token categories, the patterns, their order, and the final classification are established in the project. FLEX is not used in the implementation.

2.2 Token and Lexeme

A *token* is a category that represents a lexical unit, while a *lexeme* is the actual sequence of characters found in the source text. Consider the input

```
int a = 10;
```

The lexeme `int` belongs to the *keyword* category, `a` is an *identifier*, `=` is an *operator*, `10` is a *constant*, and `;` is *punctuation*. Therefore, the complete input contains five tokens.

The visible token categories used by the project are summarized in Table 1.

Table 1: Token categories used by the Jacaranda lexer.

Category	Description
Keyword	Reserved word recognized by the Jacaranda lexer. The set is C-oriented and also includes configured project entries such as <code>print</code> and <code>printf</code> .
Identifier	User-defined name beginning with a letter or underscore and followed by letters, digits, or underscores.
Operator	Assignment, arithmetic, comparison, or logical operator supported by the lexical rules.
Constant	Integer, real number, Boolean value, or string value. The words <code>true</code> and <code>false</code> are classified as Boolean constants, not keywords.
Punctuation	Delimiter or structural symbol such as parentheses, braces, brackets, commas, semicolons, and the colon used after <code>case</code> and <code>default</code> .

2.3 Regular Expressions

Regular expressions are a compact formal notation for describing sets of strings. They are useful in lexical analysis because many token categories can be represented as regular languages. For example, an identifier can be described by a rule that requires a letter or underscore as the first character and allows letters, digits, or underscores afterward [1].

The principal patterns used by Jacaranda are shown below. They correspond to the current implementation and therefore describe only the lexical subset supported by the project.

KEYWORD	(auto break case char const continue default do double else enum extern float for goto if int long register return short signed sizeof static struct switch typedef union unsigned void volatile while _Packed print printf bool)(?![a-zA-Z0-9_])
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*
REAL	[0-9]+\.[0-9]+
INTEGER	[0-9]+
BOOLEAN	(true false)(?![a-zA-Z0-9_])

STRING	"([^\\"\\] \\.)*"
OPERATOR	== != <= >= \+ \+ -- && \ \ \+ - * / % = < > !
PUNCTUATION	[;,:(){}\[\]]
IGNORED	\s+ and supported // line comments

Some patterns overlap, so rule priority is important. Keywords are registered before identifiers so that a reserved word such as `switch` is not classified as a normal identifier. Similarly, multi-character operators are considered before shorter alternatives, preventing an operator such as `>=` from being interpreted as two separate symbols.

The implementation is case-sensitive, following the behavior expected from C source text. Therefore, `int`, `switch`, and `while` match keyword rules, while forms such as `INT`, `SWITCH`, and `WHILE` are valid identifier lexemes under the current rules. The same distinction applies to Boolean constants: `true` and `false` are constants, while `TRUE` and `FALSE` are identifiers.

Jacaranda is a C-oriented educational lexer rather than a complete implementation of the C lexical specification. Most of the reserved words used by the analyzer were selected from the C keyword list provided by IBM [4]. The project also keeps `printf`, `bool`, and `_Packed` in its configured keyword set. The keyword `print` is included because the activity explicitly requires it to be recognized as a keyword.

2.4 Finite Automata

A finite automaton represents recognition through a finite collection of states and transitions. Starting at an initial state, the automaton consumes one input symbol at a time and moves according to the available transitions. If the complete lexeme ends in an accepting state, the sequence belongs to the represented language.

The automata included in this report are conceptual models created from the lexical rules used by Jacaranda. They are not a second implementation of the lexer. Their purpose is to visualize the recognition logic that is later expressed with regular expressions and registered in RPLY.

2.5 Lexical Errors

A lexical error occurs when the scanner reaches a sequence that cannot be matched by any of the defined token patterns. In the current implementation, RPLY reports the source position associated with the error, allowing the application to display the line and column where recognition stopped.

A file-access problem is handled separately. If the requested file cannot be opened, the application displays an input error instead of treating the file name as source code. Keeping these cases separate makes it possible to distinguish a problem with the input source from a problem with the lexical structure of the program.

2.6 Context-Free Grammar and Left Factoring

A context-free grammar (CFG) describes how terminal and non-terminal symbols can be combined to form valid structures. Although CFGs belong mainly to syntactic analysis, the activity requests a basic proposal based on one of the strings processed by the lexer.

The example `int a = 10;` can be generalized so that a declaration may appear either with an initial value or without one:

$$\begin{aligned} S &\rightarrow \text{int } ID = \textit{CONSTANT} ; \\ S &\rightarrow \text{int } ID ; \end{aligned}$$

Both productions begin with the common prefix `int ID`. Applying left factoring gives:

$$\begin{aligned} S &\rightarrow \text{int } ID A \\ A &\rightarrow = \textit{CONSTANT} ; \mid ; \end{aligned}$$

The factored form delays the decision until the point at which the two alternatives differ. For the input `int a = 10;`, the first alternative of A is selected.

3 Development

3.1 General Project Logic

The project keeps the lexical rules independent from the way the user interacts with the program. The terminal version receives a source string or a file path through a menu, while the graphical version allows the user to write code directly or select a supported source file. In both cases, the complete source text is sent to the same lexical analyzer.

The lexer processes the input from left to right using the configured rules. Recognized lexemes become tokens, whitespace and supported line comments are ignored, and an unsupported symbol produces a lexical error. Figure 1 shows the workflow of the terminal version together with the lexical-analysis path shared by both interfaces.

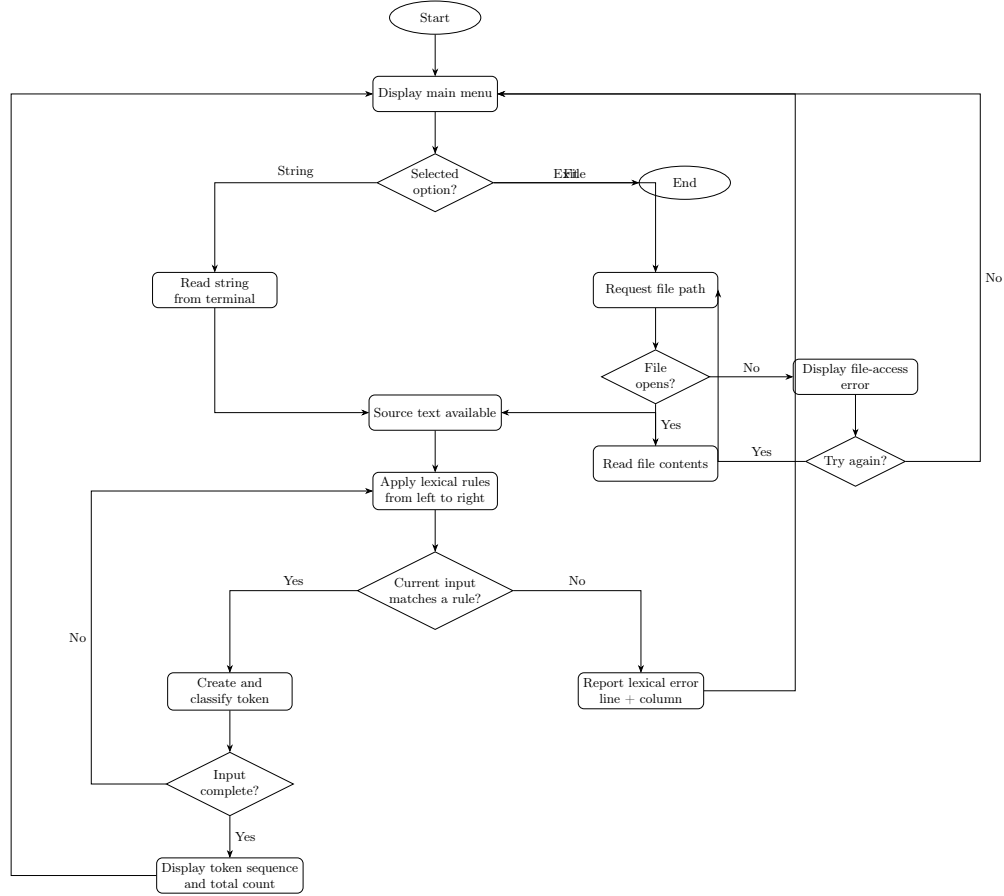


Figure 1: Workflow of the terminal version and the shared lexical-analysis process.

3.2 Project Organization and Data Flow

The implementation is divided into three main Python modules. `LexicalAnalyzer.py` contains the token rules and builds the RPLY lexer. The module `main.py` provides the terminal version and contains helper functions used to classify tokens and calculate lexical-error positions. Finally, `gui.py` implements the Tkinter graphical interface and reuses both the lexer and the helper functions instead of creating a second analyzer.

This organization keeps the lexical logic separate from the user interface. The terminal and graphical versions therefore use the same recognition rules and produce the same token categories. Figure 2 shows the role of the three modules. The graphical interface also imports the classification and position helpers from `main.py`.

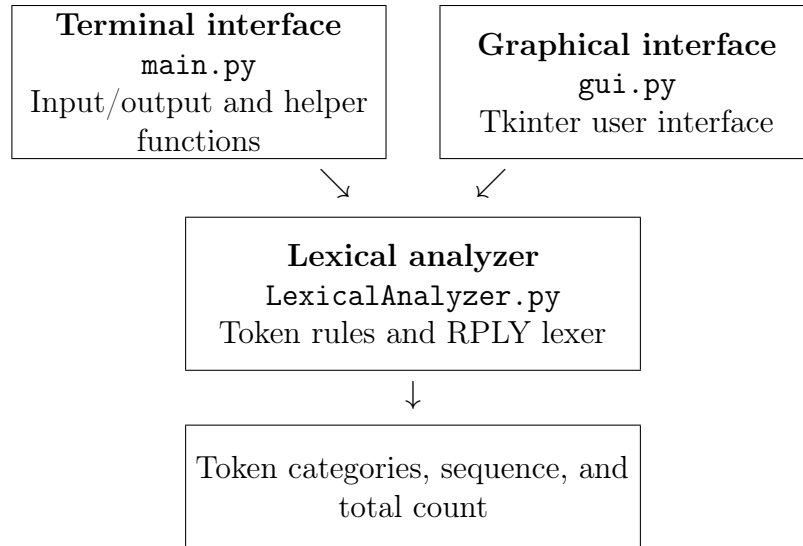


Figure 2: Organization and data flow of the current Jacaranda project.

3.3 Graphical User Interface

A graphical user interface was added to make the lexical analyzer easier to use. It was developed with Tkinter and uses the same lexical rules as the terminal version, so the interface changes the way the user interacts with the program without changing the analysis itself.

The window is divided into two adjustable panels. The *Source code* panel is located on the left and allows the user to write or load source code. The *Analysis result* panel is located on the right and shows the recognized lexemes, their categories, the token sequence, and the total number of tokens. The divider between the panels can be moved to give more space to either side.

The interface includes the buttons *Analyze*, *Open File*, and *Clear*. It also reports lexical errors directly in the result panel with the line and column where the problem was found. The visual design follows the Jacaranda identity used by the team, with purple and light lavender colors, a jacaranda flower as the application icon, and a monospaced font for source code and results.

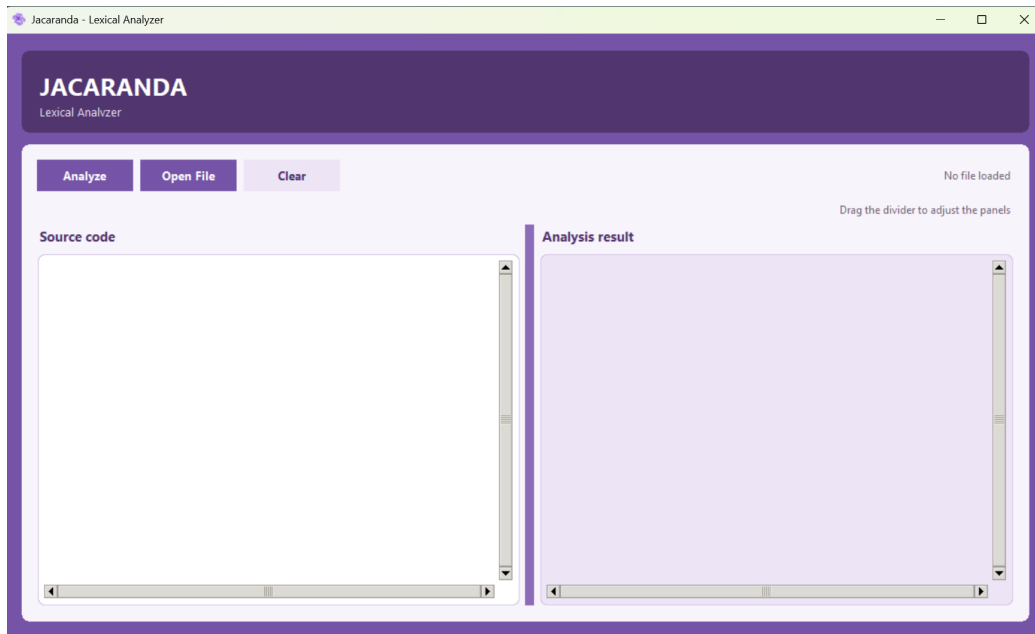


Figure 3: Graphical user interface of the Jacaranda lexical analyzer.

3.4 Finite Automata Design

To connect the implementation with the concepts studied in class, finite automata were prepared for the principal token groups. An incoming arrow identifies the initial state and a double circle identifies an accepting state. The diagrams represent only the lexical subset implemented in Jacaranda rather than the complete lexical specification of the C language.

3.4.1 Keyword Automaton

The keyword set was expanded using a C-oriented reserved-word list as a reference [4]. The lexer can recognize words including `auto`, `break`, `case`, `char`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extern`, `float`, `for`, `goto`, `if`, `int`, `long`, `register`, `return`, `short`, `signed`, `sizeof`, `static`, `struct`, `switch`, `typedef`, `union`, `unsigned`, `void`, `volatile`, and `while`. The project configuration also recognizes `_Packed`, `print`, `printf`, and `bool` as keywords.

Figure 4 shows these words as paths from a common initial state, accepting states correspond to complete words. The implemented regular expression additionally checks that a matched keyword is not immediately followed by a letter, digit, or underscore, so this prevents a lexeme such as `printer` from being divided into the keyword `print` plus another fragment.

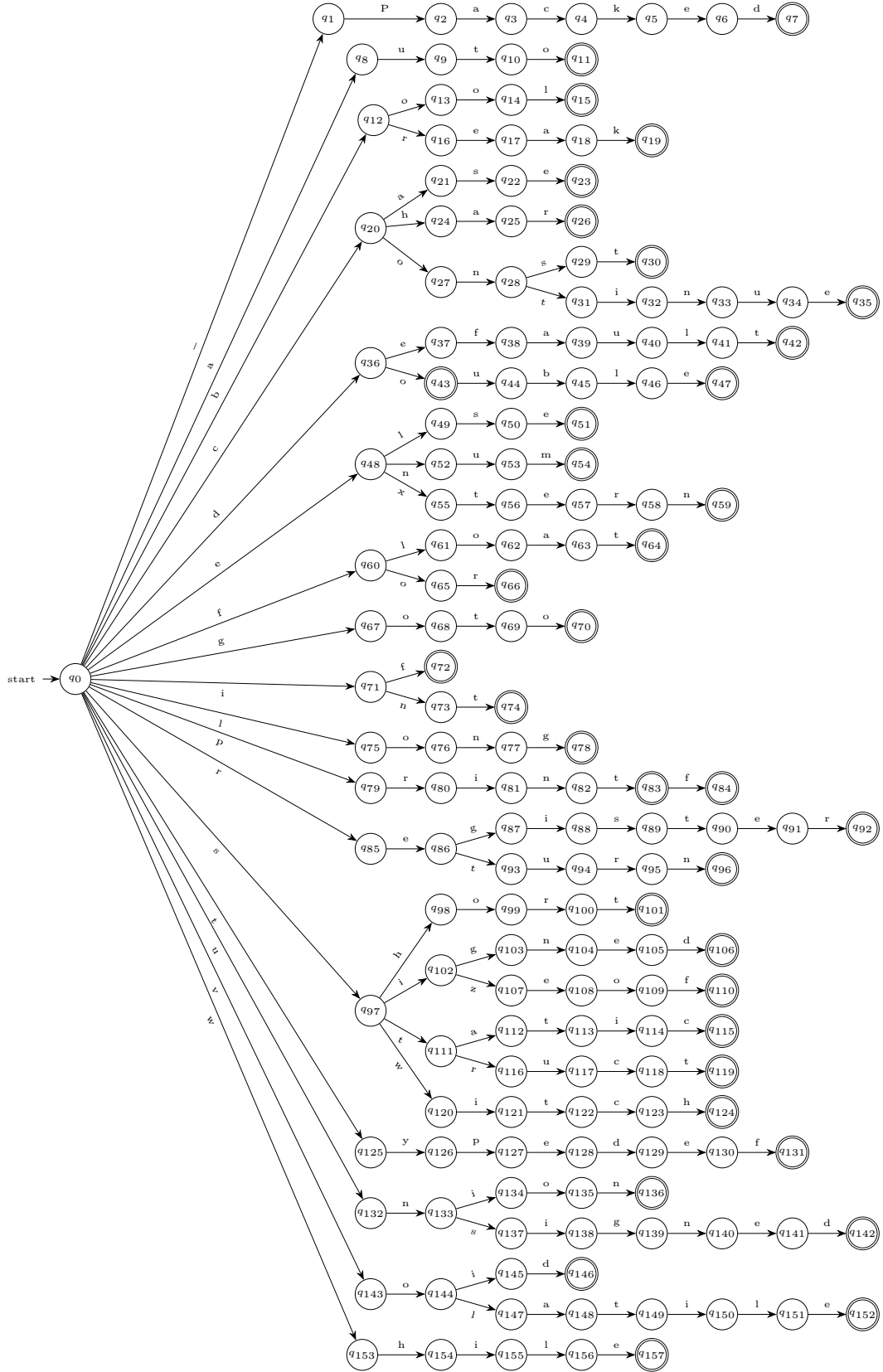


Figure 4: Finite automaton representing the keyword set supported by Jacaranda.

3.4.2 Identifier Automaton

Identifiers must begin with a letter or underscore. After the first character, letters, digits, and underscores may appear repeatedly. This behavior is represented by a transition to an accepting state followed by a loop, as shown in Figure 5.

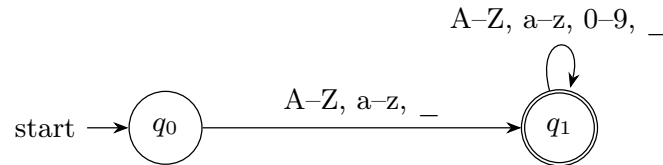


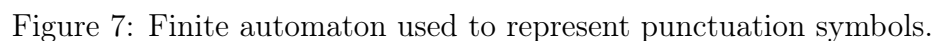
Figure 5: Finite automaton used to recognize identifiers.

3.4.3 Operator Automaton

The lexer accepts both one-character and two-character operators. Symbols such as `+`, `-`, `=`, `!`, `<`, and `>` already form valid operators after the first character, but they may continue to `++`, `-`, `==`, `!=`, `<=`, or `>=`. The logical operators `&&` and `||` require the second repeated character before reaching an accepting state.



Punctuation symbols are single-character tokens. From the initial state, any supported punctuation symbol reaches the same accepting state. Jacaranda recognizes semicolons, commas, colons, parentheses, braces, and brackets. The colon is required for C-oriented structures such as `case 1:` and `default::`.



3.4.5 Numeric Constants Automaton

The lexer distinguishes integer and real-number patterns internally, although both appear to the user under the general category *constant*. An integer contains one or more digits. A real number begins with the same digit sequence, followed by a decimal point and at least one digit after that point.

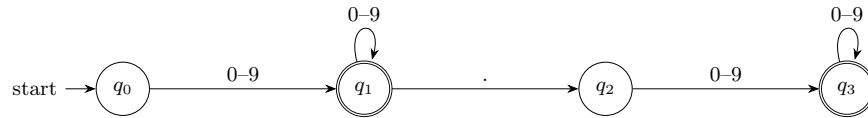


Figure 8: Finite automaton representing integer and real constants.

3.4.6 Boolean Constants Automaton

Boolean constants are the words **true** and **false**. They are internally recognized by a specific rule and are later displayed under the general category *constant*. Like keywords, their lexical pattern includes a boundary condition so that a longer identifier is not accidentally split.

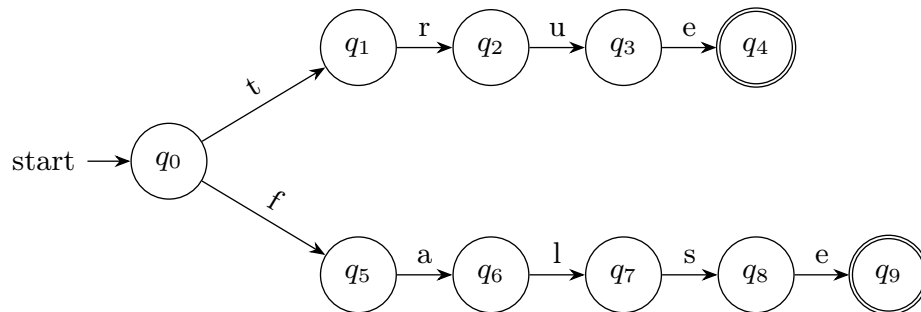


Figure 9: Finite automaton representing Boolean constants.

3.4.7 String Literal Automaton

String constants begin and end with a double quotation mark. While the automaton is inside the string, ordinary characters keep it in the content state. A backslash introduces an escaped character and then returns to the content state. A closing quotation mark reaches the accepting state.

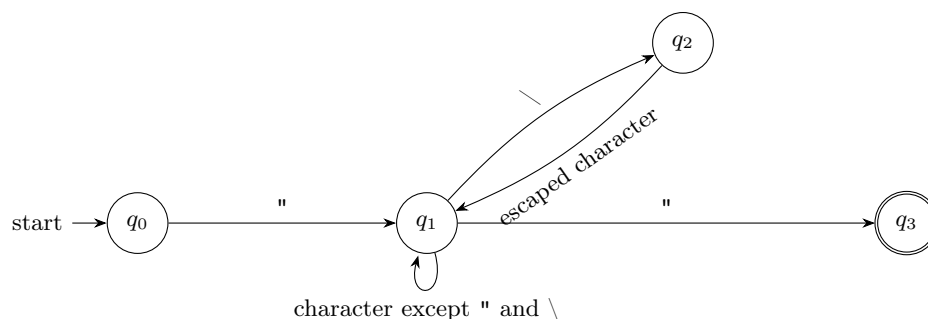


Figure 10: Finite automaton representing string constants.

3.5 Relationship Between Automata and Regular Expressions

The automata and regular expressions describe the same recognition rules from two different perspectives, and the automata make the state transitions visible, while the regular expressions provide a compact notation that can be registered in the lexer. For example, the identifier automaton accepts a first character from the set of letters or underscore and then remains in an accepting state while letters, digits, or underscores continue to appear. This is equivalent to the expression `[a-zA-Z_][a-zA-Z0-9_]*`.

The same relationship can be observed in numeric constants and string literals. For this reason, the automata are used as the theoretical representation of the lexical design, while RPLY executes the corresponding patterns in the program.

3.6 Lexer Implementation

At the implementation level, lexical recognition remains centralized in `LexicalAnalyzer.py`. The terminal version in `main.py` and the graphical interface in `gui.py` both reuse this analyzer. The GUI also reuses the token-classification and error-position helper functions from `main.py`. This prevents the project from maintaining different lexical rules for each interface.

The recognition rules are registered in a deliberate order. Keywords appear before identifiers because both can begin with letters. String and numeric constants have their own rules, and multi-character operators are written before their one-character alternatives. Whitespace and supported line comments are ignored because they do not contribute tokens to the required output.

Internally, Jacaranda distinguishes strings, real numbers, integers, and Boolean values. During presentation, all of them are mapped to the general category *constant*. This allows the implementation to keep precise recognition rules while producing the simpler output requested in the activity.

3.7 Whitespace and Case Sensitivity

Whitespace is ignored with the rule `\s+`. Consequently, spaces, tab characters, and line breaks that appear between lexemes do not generate tokens. For example, `int a=10;` and `int a = 10;` produce the same token sequence. A line break is not required after a colon or between two tokens as long as the lexical boundaries remain clear. Spaces that occur *inside* a string literal are preserved because the complete quoted text is recognized as one string constant.

The lexer is case-sensitive. Lowercase words that exactly match a configured keyword or Boolean constant are classified according to those rules, while different capitalization is processed by the identifier rule when valid. This behavior is intentional for a C-oriented lexer and is useful when testing examples such as `switch`, `case`, `true`, and their uppercase variants.

3.8 Input and Error Handling

The project supports direct text input and file input. In the terminal version, the user can enter a source string or provide a file path. In the graphical interface, source code can be written directly in the *Source code* panel or loaded with the *Open File* option.

The graphical interface limits file selection to the formats used by the project: `.txt`, `.c`, and `.h`. File problems are handled before the content is sent to the lexer. An unsupported extension, invalid UTF-8 text, or an access problem generates a clear message instead of exposing the internal Python exception. The terminal version keeps its own recovery flow for invalid paths.

Lexical errors are handled separately from file errors. When the lexer finds an unsupported symbol, the application reports the symbol together with its source line and a one-based column number. The graphical interface also displays the source line and places a caret under the position where recognition stopped.

The lexer performs lexical classification only. It does not determine whether the complete token sequence is syntactically valid C. For example, a sequence may be divided into valid tokens even if a later parser would reject its grammatical structure.

3.9 CFG Proposal and Left Factoring

The CFG proposal is based on the example `int a = 10;` and generalizes it to declarations with or without initialization. Before left factoring, the productions are:

$$\begin{aligned} S &\rightarrow \text{int } ID = \text{CONSTANT} ; \\ S &\rightarrow \text{int } ID ; \end{aligned}$$

The common prefix is extracted as follows:

$$S \rightarrow \text{int } ID \ A$$

$$A \rightarrow = \text{CONSTANT} \ ; \ ;$$

Figure 11 shows the transformation visually. The parser first consumes the shared prefix `int ID`; the auxiliary non-terminal A then determines whether the declaration contains an initialization or ends immediately.

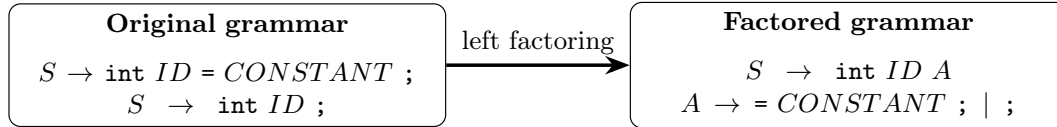


Figure 11: Left factoring applied to the declaration example.

3.10 Test Cases

The tests combine the examples specified in the activity with additional cases that exercise the expanded keyword set, punctuation, whitespace handling, case sensitivity, file input, and error reporting. Table 2 summarizes the principal cases.

Table 2: Main tests used for the Jacaranda lexical analyzer.

Input	Purpose / expected behavior	Tokens
<code>int a = 10;</code>	Verify the categories keyword, identifier, operator, constant, and punctuation.	5
<code>printf("This is an example");</code>	Verify keyword recognition, a complete string constant, and punctuation.	5
<code>print("Hello world");</code>	Verify that <code>print</code> is a keyword and that spaces inside the string remain part of the same constant.	5
<code>switch(option){case 1: break;}</code>	Verify <code>switch</code> , <code>case</code> , <code>break</code> , braces, and colon punctuation.	11
<code>int A = 10;</code>	Verify that repeated spaces are ignored and uppercase letters are valid in identifiers.	5
<code>true false TRUE</code>	Verify Boolean constants and case sensitivity; <code>TRUE</code> is an identifier.	3
<code>40_tokens.txt</code>	Exercise several token categories and verify the requested total count.	40
<code>int a = 10 @ 3;</code>	Confirm that an unsupported symbol produces a lexical error with line and column information.	–
Supported file input	Confirm that <code>.txt</code> , <code>.c</code> , and <code>.h</code> files can be selected and analyzed.	–
Unreadable or invalid file	Confirm that file problems are reported separately from lexical errors.	–

The prepared 40-token file is used as the larger counting test. Whitespace and line breaks do not add tokens, so only recognized lexical units contribute to the final total. The `switch` example is useful for checking the colon because it appears naturally after `case` and `default` labels.

4 Results

The final version of the Jacaranda lexical analyzer was tested through the graphical interface using short examples, source files, and intentional errors. The following evidence shows the behavior obtained from the implementation that will be submitted.

4.1 Required Declaration Example

The input `int a = 10;` was used as the basic declaration test. The analyzer recognized `int` as a keyword, `a` as an identifier, `=` as an operator, `10` as a constant, and `;` as punctuation. The final count was 5 tokens.

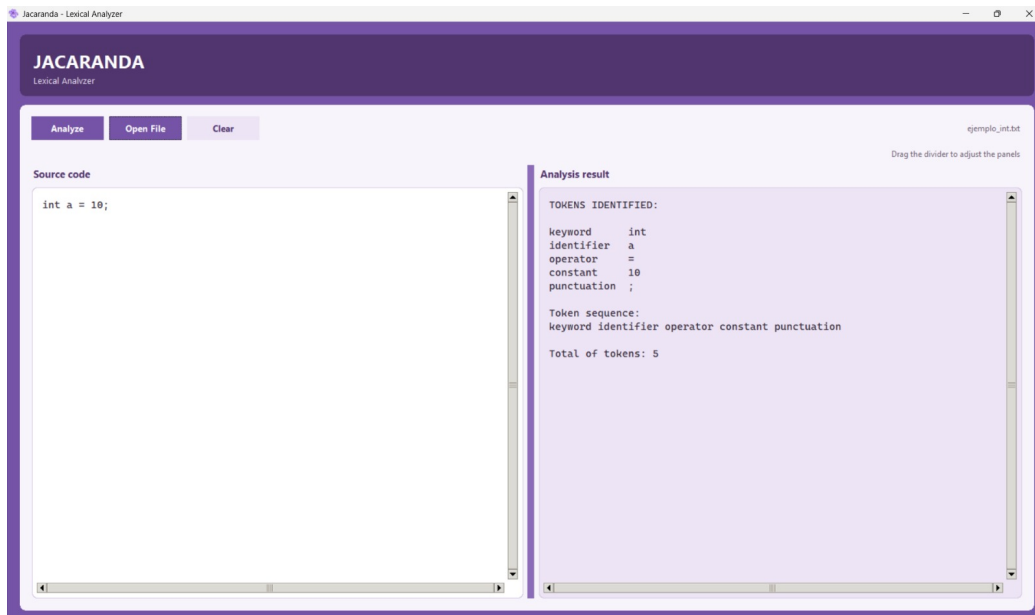


Figure 12: Recognition of the declaration example.

4.2 String and Keyword Example

Two similar inputs were tested to verify the recognition of printing keywords and string constants. In both cases, the text inside quotation marks was recognized as one constant.

The first test used `printf("Hello")`. The analyzer classified `printf` as a keyword, the quotation content as a constant, and the parentheses as punctuation.

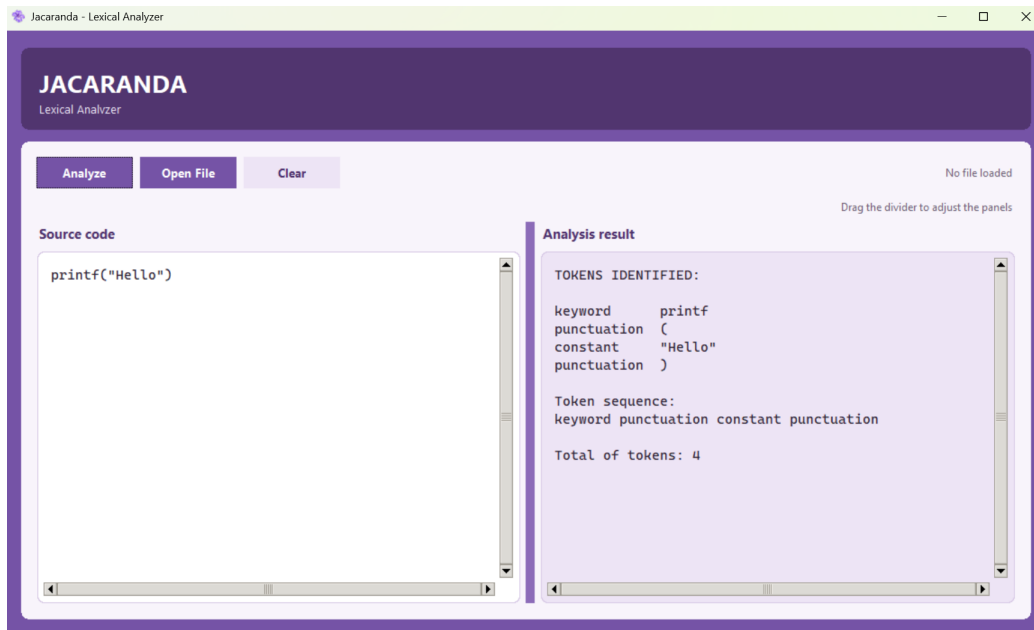


Figure 13: Recognition of `printf` and a string constant.

The second test used `print("Hello")`. In this case, `print` was also recognized as a keyword, confirming the requirement stated in the activity.

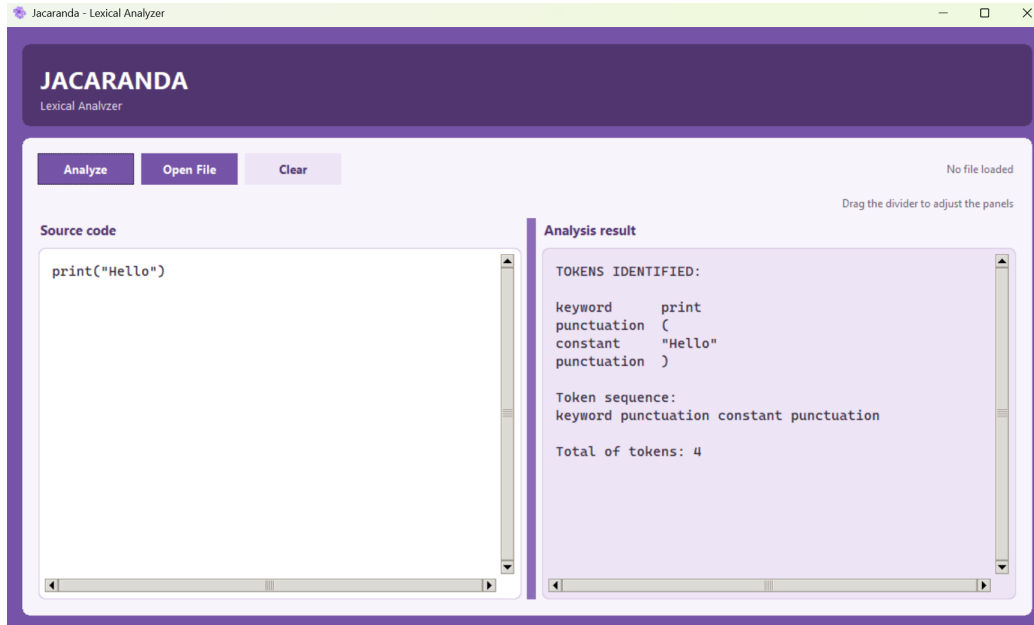


Figure 14: Recognition of `print` as a keyword and a string constant.

4.3 Switch and Colon Test

A `switch` structure was tested to verify the expanded keyword set and the colon punctuation rule. The analyzer recognized words such as `switch`, `case`, `default`, `break`, and `print` as keywords. The colons following `case 1` and `default` were classified as punctuation without producing a lexical error.

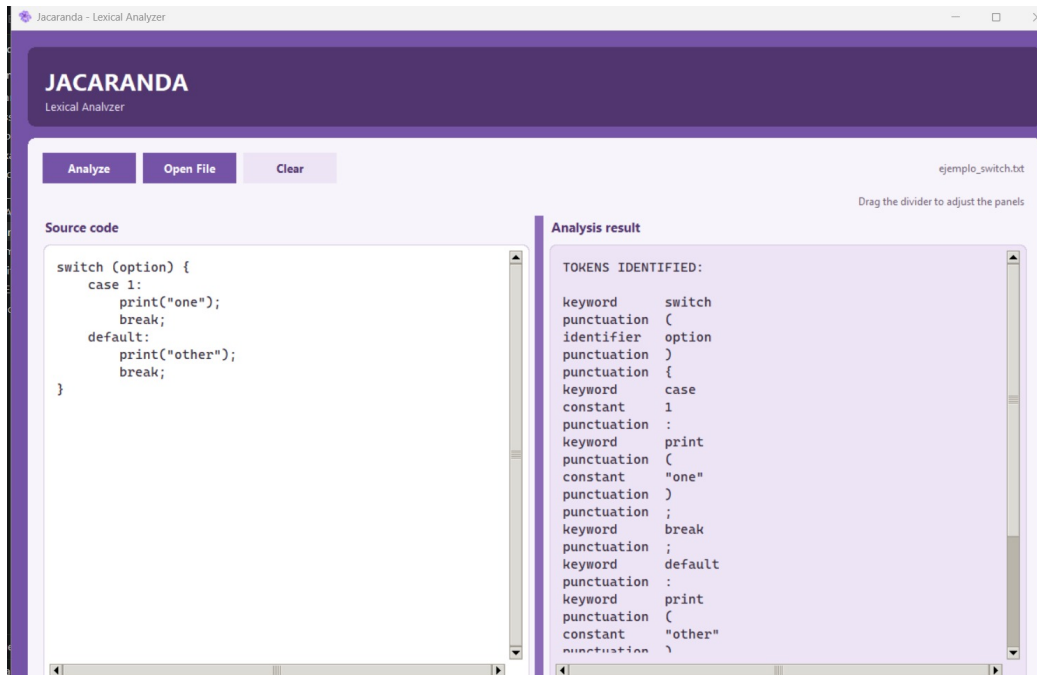


Figure 15: Recognition of `switch`, `case`, `default`, and colon punctuation.

4.4 Forty-Token Test

The file `40_tokens.txt` was used as a larger test for the token counter. It combines several keywords, identifiers, operators, constants, and punctuation symbols. Figure 16 shows the source code and the first part of the recognized token list.

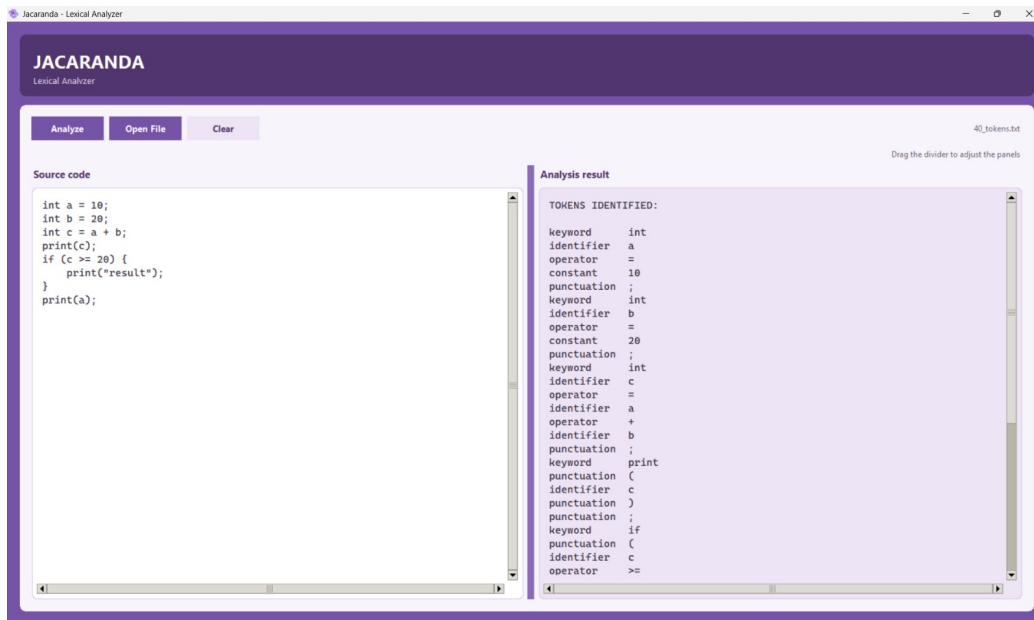


Figure 16: Analysis of the 40-token test file.

Because the complete list does not fit in a single view, the result panel was scrolled to show the final part of the analysis. Figure 17 confirms that exactly 40 tokens were recognized.

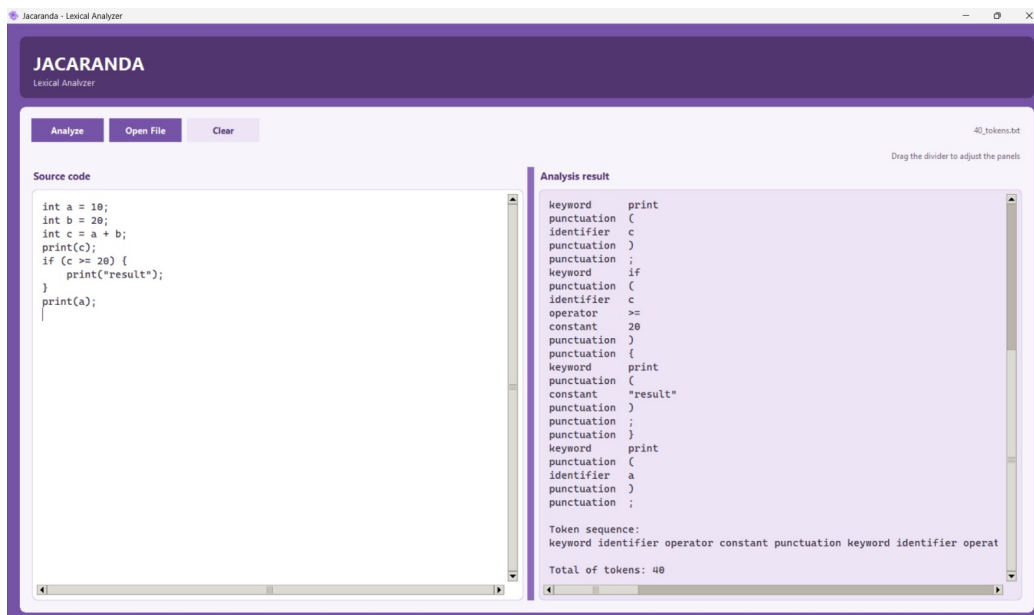


Figure 17: Final result of the 40-token test showing a total of 40 tokens.

4.5 Lexical Error Test

An unsupported symbol was introduced intentionally with the input `int a = 10 @ 3;`. The analyzer stopped when it found `@` and reported a lexical error at line 1, column 12. The graphical interface also marked the position with a caret, making the location of the error easier to identify.

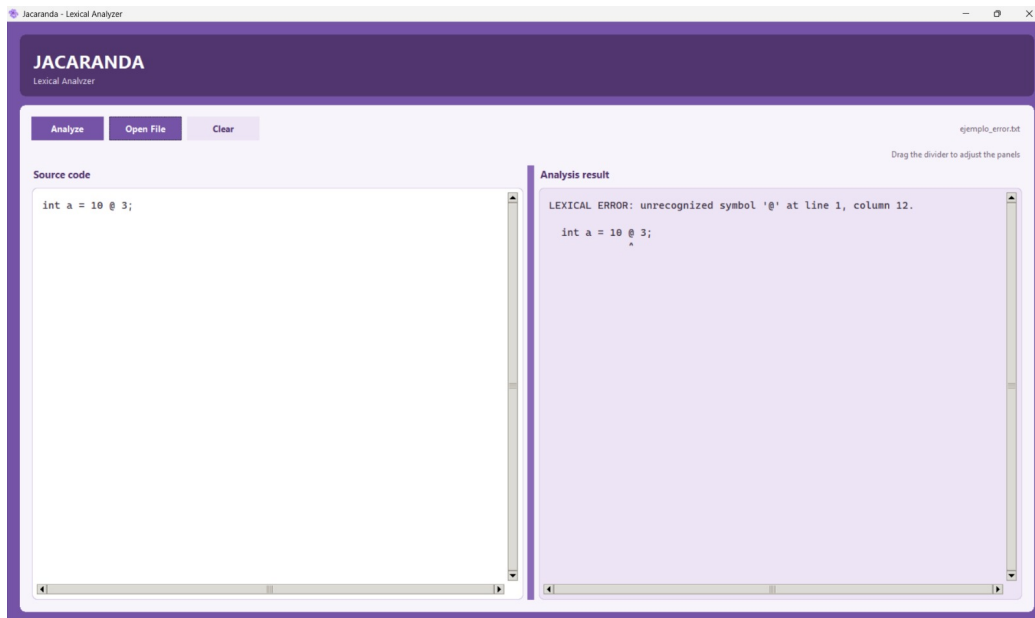


Figure 18: Detection and location of an unsupported lexical symbol.

4.6 File Input Test

File input was tested through the *Open File* option. The graphical selector filters the source formats supported by the project, as shown in Figure 19, the available categories correspond to `.txt`, `.c`, and `.h` files.

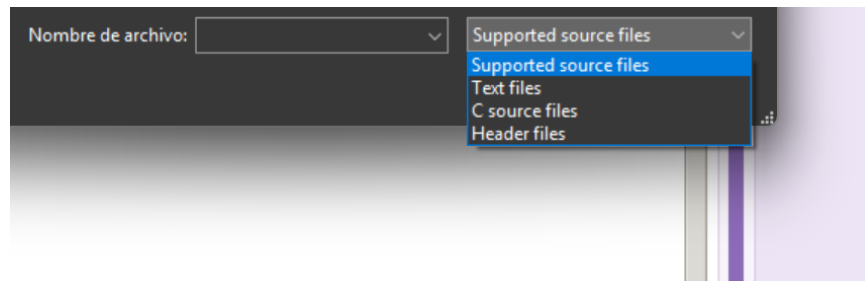


Figure 19: File-selection window showing the supported source file formats.

A successful file test was performed with `ejemplo_espacios.txt`, after selection, its content was loaded into the source-code panel. Then, after pressing *Analyze*, the code

was processed with the same lexer used for direct text input, extra spaces between lexemes were ignored, while spaces inside the quoted string remained part of the string constant, and as you can see, the execution produced 10 tokens.

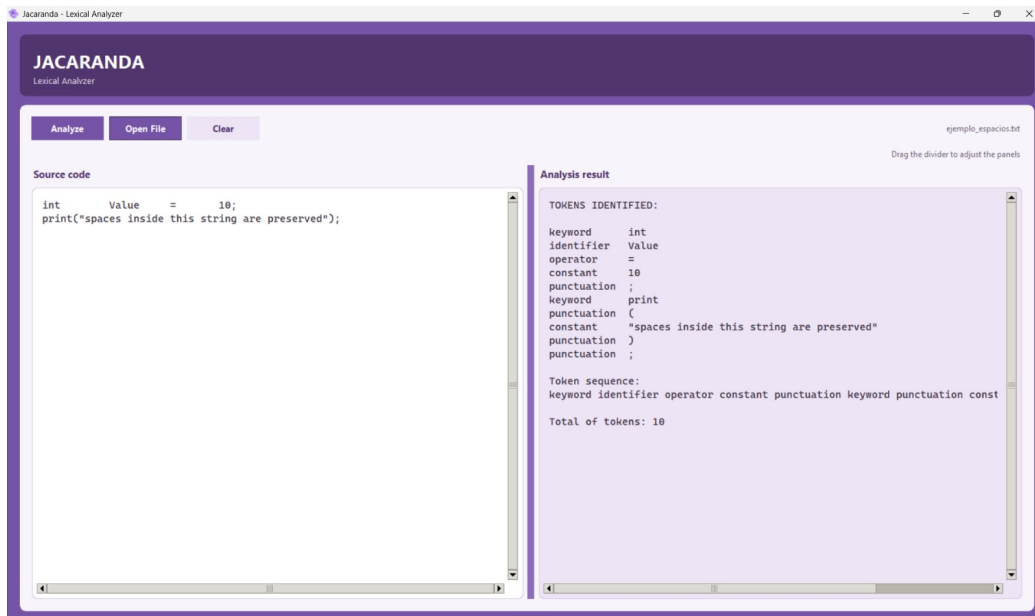


Figure 20: Execution of the lexical analyzer using a text file as input.

5 Repository

Repository link: github.com/ErickaAgui/unam.fi.compilers.g5.01

6 Conclusions

Aguilar Lara Ericka. This first part of the project helped me understand how the theoretical concepts of lexical analysis can be transformed into a functional software tool. Throughout the development of Jacaranda, I was able to see the relationship between regular expressions, finite automata, tokens, and lexemes, and how all of them work together to recognize and classify source code correctly. I also learned how important details such as rule priority, case sensitivity, whitespace handling, and lexical error detection are when designing a lexer. The different test cases allowed us to verify that the analyzer correctly recognizes keywords, identifiers, operators, constants, and punctuation, while also reporting errors clearly. Finally, keeping track of the team's progress and making sure that the different parts of the project were properly integrated showed me the importance of communication and planning in software development.

Bautista Reyes Sofia. The development of this project allowed me to better

understand the relationship between finite automata and regular expressions. The automata show the recognition process step by step, while regular expressions represent the same lexical rules in a much more compact form. Comparing both representations made it possible to understand how lexical rules can first be described theoretically and then implemented in a working program.

For example, an automaton makes each state and transition visible, while a regular expression can describe the complete pattern in only one line. Working with both approaches helped me understand more clearly how the theoretical concepts of lexical analysis are connected with the actual behavior of the lexer and how these representations can be used together during the design of a lexical analyzer.

Cruz Basilio Ximena Carolina. As the Model Designer, this project helped me understand lexical analysis much better because I was able to connect the code with the diagrams we studied in class. Creating the finite automata in TikZ helped me see how the lexer recognizes different tokens by following states and transitions. Before working on this project, I understood the concepts mostly in theory, but seeing how the regular expressions in the code could be represented visually made everything easier to understand.

I also learned that the diagrams have to be consistent with the real implementation of the lexer, especially with keywords, identifiers, constants, operators, and punctuation. In general, this project gave me a clearer idea of how the first stage of a compiler works and how modeling can make a program easier to understand and explain.

Jardón Marín Rodrigo. In this part of the project, I correctly understood the operation of a lexical analyzer and its components, as well as its internal logic. Relating grammatical and automata theory to the developed code was key to correctly classifying tokens and accurately identifying lexical errors when running the test cases. Furthermore, the use of regular expressions and the implementation of good software engineering practices through Git and GitHub allowed us to work collaboratively and in an organized manner. This whole process leaves me with a solid, functional foundation ready to be integrated into the next stage: the syntax analyzer.

Garibay Zamorano Josué Benjamín. The project showed the relationship between lexical analysis and the following stages of a compiler. The lexer identifies the basic units of the source code, while the CFG and left-factoring example illustrate how those tokens can later be processed by a parser. The organization of the source code, tests, documentation, and repository also demonstrated the importance of maintaining a clear project structure during software development.

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson Addison-Wesley, 2006.

- [2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Morgan Kaufmann, 2022.
- [3] RPLY, “RPLY Documentation.” [Online]. Available: <https://rply.readthedocs.io/en/latest/>. Accessed: Oct. 5, 2026.
- [4] IBM, “C reserved keywords.” [Online]. Available: <https://www.ibm.com/docs/es/debug-for-zos/15.0.x?topic=programs-c-reserved-keywords>. Accessed: Oct. 5, 2026.